

Technical Report - System Architect Lab

Linux, CPU Scheduling and Docker Containerization

LIU 2026 - Introduction to Operating Systems

Equipe / Team: Ahmed Cheikh 12530219 and Varha Mohamed Mahmoud 12530130

GitHub repository: <https://github.com/Spilled-mcoffee/System-Architect-Lab>

- Shell automation with Bash
- CPU scheduling simulation: Round Robin and SRTF
- Docker infrastructure: custom Debian image, MySQL, Apache and automated CRUD

Introduction

This report presents the implementation of the System Architect Lab mini-project. The project combines three important dimensions of modern operating systems: Linux administration through Bash scripting, CPU scheduling policy simulation, and deployment of a containerized infrastructure using Docker.

The adopted approach organizes each component into a separate folder, automates repetitive operations, and tests the technical elements inside a reproducible Docker environment. The Bash and Python scripts are executed inside a custom Debian container, avoiding risky modifications on the host machine.

Contribution Matrix

The following distribution presents the main responsibilities of each member. Both members contributed to testing, verification and preparation of the final deliverables.

Member	Main responsibilities	Contribution
Ahmed Cheikh	Bash automation, Docker Compose configuration, MySQL/Apache integration, deployment testing, documentation.	50%
Varha Mohamed Mahmoud	CPU scheduling simulation, result verification, CRUD automation, report validation and demonstration preparation.	50%

1. Shell Automation with Bash

1.1 Objective

The first part of the project consists of developing an administration script named `admin_systeme.sh`. This script automates user account creation, workspace creation, ownership assignment and access permission configuration.

1.2 Script behavior

- Checks administrator privileges before execution.
- Reads the username from the terminal.
- Verifies whether the account already exists to avoid duplicates.
- Creates the user with a home directory.
- Creates a workspace directory inside the user's home directory.
- Applies `chown` to set ownership and `chmod 700` to restrict access to the owner.
- Simulates disk quota assignment to illustrate the concept without modifying advanced filesystem configuration.

1.3 Execution inside the Debian container

To make testing safer and more reproducible, the Bash script is not executed directly on the host machine. The local `bash/` folder is mounted inside the Debian container at `/lab/bash`. Therefore, users and directories created by the script exist only inside the isolated container environment.

```
docker exec -it debian_lab bash
ls -la /lab/bash
bash /lab/bash/admin_systeme.sh
id testuser
ls -ld /home/testuser/workspace
```

The expected result is the creation of a test user, a `/home/testuser/workspace` directory and permissions similar to `drwx-----`, corresponding to mode 700.

2. CPU Scheduling Modeling

2.1 Objective

The second part compares two scheduling algorithms: Round Robin and Shortest Remaining Time First (SRTF). The simulation calculates waiting times, turnaround times and produces a Gantt-style representation of execution.

2.2 Simulation data

Process	Arrival time	Burst time
P1	0	7
P2	2	4
P3	4	1
P4	5	4

2.3 Round Robin

Round Robin assigns each process a fixed time quantum. In this implementation, the selected quantum is 2 time units. If a process is not completed after its quantum, it is placed at the end of the ready queue. This algorithm promotes fairness, but it may increase average waiting time due to frequent context switches.

2.4 SRTF

SRTF selects, at every instant, the process with the shortest remaining time. It is preemptive and may interrupt a running process when a shorter process arrives. This algorithm tends to reduce average waiting time, but it may penalize long processes.

2.5 Execution inside Docker

The Python script `scheduling.py` is also executed inside the Debian container. The `scheduling/` folder is mounted at `/lab/scheduling`. Since the custom Debian image already contains Python, no manual package installation is needed during the demonstration.

```
docker exec -it debian_lab bash
python3 --version
python3 /lab/scheduling/scheduling.py
```

3. Docker Infrastructure and Containerization

3.1 General architecture

The third part of the infrastructure is based on Docker Compose. The architecture contains three main services: a custom Debian container, a MySQL server and an Apache HTTP server.

Service	Role	Main configuration
debian_lab	Linux testing environment	Custom image built with <code>Dockerfile.debian</code>
mysql_server	etudiants database	MySQL 8.0, host port 3307 to container port 3306, persistent volume
apache_server	HTTP server	httpd image, host port 8080 to container port 80, custom web page

3.2 Custom Debian image

A custom Debian image was created to avoid repeatedly installing Python and administration tools each time the container is recreated. `Dockerfile.debian` starts from `debian:stable-slim` and installs `python3`, `passwd`, `procps`, `iproute2`, `iputils-ping` and `nano`.

```
FROM debian:stable-slim
```

```
RUN apt-get update && apt-get install -y --no-install-recommends \
    python3 passwd procps iproute2 iputils-ping nano \
    && rm -rf /var/lib/apt/lists/*
```

```
WORKDIR /lab
```

```
CMD ["bash"]
```

In docker-compose.yml, the debian_lab service uses build to create this local image. The bash/ and scheduling/ folders are mounted into the container so the project scripts can be executed directly.

```
debian_lab:
  build:
    context: .
    dockerfile: Dockerfile.debian
  image: system-architect-debian:1.0
  container_name: debian_lab
  tty: true
  stdin_open: true
  volumes:
    - ../bash:/lab/bash
    - ../scheduling:/lab/scheduling
```

3.3 MySQL, CRUD and persistent volume

The MySQL service initializes a database named etudians. Internal port 3306 is exposed on host port 3307 to avoid conflict with an existing local MySQL server. CRUD operations are automated by a Bash script that sends sql/crud.sql to the MySQL container.

```
./scripts/run_crud.sh
```

The mysql_data volume is mounted on /var/lib/mysql. This path is the internal location where MySQL stores its data files. Thanks to this volume, data can persist even if the container is stopped or recreated. docker compose down keeps the volume, while docker compose down -v removes it.

3.4 Apache server and custom page

The Apache service uses the official httpd image. Container port 80 is exposed on host port 8080. The local web/ folder is mounted at /usr/local/apache2/htdocs so that Apache displays a custom HTML page instead of the default page.

```
apache_server:
  image: httpd:latest
  container_name: apache_server
  ports:
    - "8080:80"
  volumes:
    - ../web:/usr/local/apache2/htdocs
```

Verification is performed in the browser at <http://localhost:8080>.

3.5 Test protocol

1. Build and start the infrastructure with `docker compose up -d --build`.
2. Verify the containers with `docker compose ps`.
3. Execute the Bash script inside the Debian container.
4. Execute the Python scheduling simulator inside the Debian container.
5. Open the Apache page in the browser at <http://localhost:8080>.
6. Run the CRUD operations with `./scripts/run_crud.sh`.
7. Verify that MySQL, Apache and Debian remain operational.

Conclusion

The project provides a coherent integration of Linux administration, scheduling simulation and containerization. The custom Debian image improves reproducibility because the required tools are built into the image. Executing the Bash and Python scripts inside the container improves testing safety and demonstrates Docker's role as a controlled environment. Finally, the combination of MySQL, a persistent volume, an Apache server and automation scripts provides a clear, testable and presentable infrastructure.